

Notebook

February 8, 2026

1 ECDSA/DSA с Nonce со Статистическим Отклонением

NB! Эту записную книжку надо открывать при помощи sagemath в режиме jupyter, а не с использованием обычного python, так как ему необходимы API sagemath. Установите [Sagemath](#) и запустите:

```
sage -n jupyter
```

Мы прошли, что имплементации DSA и ECDSA становятся уязвимыми, если:

1. Использовать предсказуемые k (nonce)
2. Использовать повторяющиеся k

Также я упоминал, что использование k со статистическим отклонением также может привести к раскрытию ключа. Это упражнение сфокусировано как раз на этой проблеме.

1.1 Введение

Сначала, давайте вспомним, из чего состоят DSA и ECDSA. В случае DSA у нас есть хеш-функция $Hash(m)$, мультипликативная группа Z_p^* по модулю простого числа p и генератор g порядка q , где q - самый большой простой делитель $p-1$ (т.е. g - генератор самой большой подгруппы с порядком, равным простому числу, группы Z_p^*). Закрытый ключ - $x \in (0, q)$, а $y = g^x \bmod p$ открытый ключ.

Алгоритм подписи m :

1. Выбрать случайный nonce (number used once; число, используемое один раз) $k \in (0, q)$
2. Вычислить $r = (g^k \bmod p) \bmod q$. Если $r = 0$, то вернуться к пункту 1.
3. Вычислить $s = \frac{Hash(m) + x \cdot r}{k} \bmod q$. Если $s = 0$, то вернуться к пункту 1.
4. Вернуть (r, s) . Это и есть подпись

Чтобы проверить подпись, нам надо удостовериться, что $r, s \in (0, q)$ и следующее уравнение верно:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = r$$

В случае ECDSA дана эллиптическая кривая $E(GF(p))$, некоторая точка G на кривой в качестве генератора подгруппы простого порядка q , т.е. $q \cdot G = 0$ (Точка

на Бесконечности). Также определена хеш-функция $Hash(m)$. Чтобы создать ключи мы:

1. Выбираем случайное $d \in (0, q)$ (закрытый ключ)
2. Вычисляем точку $Q = d \cdot G$ (открытый ключ)

Алгоритм подписи m :

1. Выбрать случайный nonce (number used once; число, используемое один раз) $k \in (0, q)$
2. Вычислить $(x_1, y_1) = k \cdot G$
3. Вычислить $r = x_1 \bmod q$. Если $r = 0$, то вернуться к пункту 1.
4. Вычислить $s = \frac{Hash(m) + r \cdot d}{k} \bmod q$. Если $s = 0$, то вернуться к пункту 1.
5. Вернуть (r, s) . Это и есть подпись.

Алгоритм проверки подписи:

1. Проверить, что $r, s \in (0, q)$
2. Вычислить $(x_1, y_1) = \frac{Hash(m)}{s} \cdot G + \frac{r}{s} \cdot Q$
3. Проверить, что $r = x_1 \bmod q$

В этот раз я пропустил доказательство корректности, раз уж мы его уже прошли в прошлом задании, а в атаке оно не поможет.

Так что же мы подразумеваем под nonce со статистическим отклонением? Это значит, что распределение получаемых nonce сосредоточено в каком-то регионе. Например, 8 верхних битов k всегда равны 0. Так как $s = \frac{Hash(m) + x \cdot r}{k} \bmod q$, то

$$s \cdot k = Hash(m) + x \cdot r \bmod q$$

$$k = \frac{Hash(m) + x \cdot r}{s} \bmod q$$

$$x \cdot \frac{-r}{s} = \frac{Hash(m)}{s} - k \bmod q$$

Мы можем представить это выражение в следующем виде:

$x \cdot a = b - c \bmod q$, where $a = \frac{-r}{s} \bmod q$, $b = \frac{Hash(m)}{s}$ и $c = k$. Заметим, что c намного меньше x , a и b , и мы можем сказать, что $x \cdot a \simeq b \bmod q$.

1.2 HNP, CVP, LLL и Gram-Schmidt (Грам-Шмидт)

То, на что мы смотрим - это вариант проблемы скрытого числа (the Hidden Number Problem, HNP):

Восстановление $\alpha \in F_p$ такого, что для многих известных случайных $t \in F_p$ нам известны $MSB_{l,p}(\alpha t)$ для некоторого $l > 0$. Алгоритм для решения этой задачи

основывается на проблеме ближайшего вектора (CVP, Closest Vector Problem). Но чтобы описать её, нам надо сначала познакомиться с решётками (lattices).

Пусть $\{b_1, \dots, b_s\}$ - это множество линейно независимых векторов в $\mathbb{R}^s$.

Множество векторов

$$L = \{z | z = \sum_{i=1}^s c_i b_i, c_1, \dots, c_s \in \mathbb{Z}\}$$

называется полноранговой решёткой размерности s . Множество $\{b_1, \dots, b_s\}$ называется базисом L .

The Closest Vector Problem (CVP, Проблема Ближайшего Вектора):

При данном векторе $r \in \mathbb{R}^s$ найти вектор решётки $v \in L$ такой, что

$$\|r - v\| = \min_{z \in L} \|r - z\|$$

CVP - это NP-полная задача, но существуют полиномиальные алгоритмы, которые позволяют найти примерное решение, например, LLL (Lenstra, Lenstra, Lovasz; алгоритм Ленстры - Ленстры - Ловаса).

LLL использует процесс Грама-Шмидта (Gram-Schmidt process). Это метод ортонормирования набора векторов в пространстве с определенным скалярным умножением.

Даны $u, v \in \mathbb{R}^s$, оператор проекции определяется как $proj_u(v) = \frac{(u,v)}{(u,u)}u$, где (u,v) обозначает скалярное произведение. Для указанного набора векторов $v_1, \dots, v_k \in \mathbb{R}^s$ в Граме-Шмидте сначала выполняется ортогонализация:

$$\begin{aligned} u_1 &= v_1 \\ u_2 &= v_2 - proj_{u_1}(v_2) \\ &\dots \\ u_k &= v_k - \sum_{j=1}^{k-1} proj_{u_j}(v_k) \end{aligned}$$

Цель - получить ортогональный базис (когда скалярное произведение каждой пары различных векторов базиса равно нулю). Далее каждый вектор надо поделить на корень скалярного произведения вектора на самого себя, чтобы получить ортонормированный базис.

Алгоритм LLL вычисляет LLL-редуцированные базисы. Если дан базис

$$B = \{b_1, b_2, \dots, b_n\}$$

, ортогональный базис, полученный от него процессом Грама-Шмидта (без нормирования)

$$B^* = \{b_1^*, b_2^*, \dots, b_n^*\}$$

и коэффициенты Грама-Шмидта

$$\mu_{i,j} = \frac{(b_i, b_j^*)}{(b_j^*, b_j^*)}, \text{ для всех } 1 \leq j < i \leq n.$$

Тогда базис B LLL-редуцирован, если существует параметр $\delta \in (0.25, 1]$ такой, что выполняются следующие условия:

1. (Условие уменьшения размера) For $1 \leq j < i \leq n : |\mu_{i,j}| \leq 0.5$. Это свойство гарантирует уменьшение длины векторов у упорядоченного базиса.
2. (Условие Ловаса) Для $k = 2, 3, \dots, n : \delta \|b_{k-1}^*\|^2 \leq \|b_k^*\|^2 + \mu_{k,k-1}^2 \|b_{k-1}^*\|^2$.

По сути, δ указывает, насколько хорошо редуцирован базис. Ниже приведен псевдокод алгоритма LLL на питоноподобном синтаксе. B - это матрица, каждый ряд которой представляет отдельный вектор базиса.

```
def LLL(B, delta):
    B = copy(B)
    Q = gramschmidt(B)
    def mu(i, j):
        v = B[i]
        u = Q[j]
        return (v*u) / (u*u)

    n = len(B)
    k = 1

    while k < n:
        for j in reverse(range(k)):
            if abs(mu(k, j)) > 1/2:
                B[k] := B[k] - round(mu(k, j))*B[j]
                Q := gramschmidt(B)

        if (Q[k]*Q[k]) >= (delta - mu(k, k-1)^2) * (Q[k-1]*Q[k-1]):
            k := k + 1
        else:
            B[k], B[k-1] := B[k-1], B[k]
            Q := gramschmidt(B)
            k := max(k-1, 1)

    return B
```

Больше про LLL можно прочитать на [википедии](#). Основная идея в том, что он пытается редуцировать базис, переставляя и вычитая векторы друг из друга. В результате каждый вектор в новом базисе будет линейной комбинацией векторов в изначальном базисе.

1.3 Как решать проблему со статистическим отклонением nonce у DSA/ECDSA при помощи LLL

Вспомним, что $x \cdot a \simeq b \bmod q$, что значит, что $x \cdot a \simeq b + j \cdot q, j \in \mathbb{Z}$. Нам нужно найти x . a , b и q известны. Поэтому для нескольких элементов $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n$ мы

можем построить следующую матрицу:

$$\begin{vmatrix} q & 0 & 0 & \dots & 0 \\ 0 & q & 0 & \dots & 0 \\ 0 & 0 & q & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & q \\ a_1 & a_2 & a_3 & \dots & a_s \\ b_1 & b_2 & b_3 & \dots & b_s \end{vmatrix}$$

Каждый ряд представляет собой вектор. Нам известно, что $x \cdot \vec{a} \simeq \vec{b}$ (здесь коэффициенты в Fq), поэтому один из векторов базиса, который вычислит LLL, будет вектор $\vec{c} = \vec{b} - x \cdot \vec{a} + \sum_{i=1}^n (t_i \vec{q}_i)$, $t_i \in \mathbb{Z}$ (здесь коэффициенты в $\mathbb{Q}$). Но в итоге мы хотим вычислить x , как нам это сделать? Всё просто, дополним векторы двумя элементами. К векторам, которые содержат q добавляем два нулевых элемента. Мы хотим сбалансировать финальный вектор так, чтобы все элементы в нем были примерно одного порядка. Если отклонение в верхних l битах k , то в векторе $\vec{a}$ мы можем выставить $(n+1)$ -й в $\frac{1}{2^l} \in \mathbb{Q}$, а у $\vec{b}$ можно выставить $(n+2)$ -й элемент в $\frac{q}{2^l}$. Поскольку конечный вектор будет $\vec{b} - x \cdot \vec{a}$, то его последний элемент будет равен $\frac{q}{2^l}$, а предпоследний - $\frac{x}{2^l}$ поэтому все элементы будут примерно одного порядка.

$$\begin{vmatrix} q & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & q & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & q & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & q & 0 & 0 \\ a_1 & a_2 & a_3 & \dots & a_n & \frac{1}{2^l} & 0 \\ b_1 & b_2 & b_3 & \dots & b_n & 0 & \frac{q}{2^l} \end{vmatrix}$$

Таким образом в конце мы можем проверить все строки (векторы нового базиса) на то, не равен ли последний элемент $\frac{q}{2^l}$, и если обнаружим такой вектор, то предпоследний элемент с высокой вероятностью будет равен $\frac{-x}{2^l}$, из чего мы можем легко вычислить x .

Как мы видим, атака состоит из 4х шагов:

1. Собрать достаточное количество подписей (r, s) (оно зависит от отклонения)
2. Вычислить векторы $\vec{a}$ и $\vec{b}$ и определить решётку (матрицу)
3. Использовать LLL для решения SVP (нахождения самого короткого вектора)
4. Вычислить x из полученного базиса

Использовать LLL просто:

```
[ ]: from sage.all import *

# Рациональные числа можно использовать так:
print (Rational(1)/2)
M=matrix([[0,1,1],[2,3,1],[1,1,1]])
```

```
print (M.LLL())
```

1.4 Назад к отклонениям

Что произойдет в случае, если статистическое отклонение затрагивает нижние биты k ? Например, нижние l битов k содержат значение z : $k = 2^l \cdot k_0 + z$.

$$s = \frac{\text{Hash}(m) + x \cdot r}{k} \bmod q$$

$$k = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q$$

$$2^l \cdot k_0 + z = \frac{\text{Hash}(m) + x \cdot r}{s} \bmod q$$

$$2^l \cdot k_0 = \left(\frac{\text{Hash}(m)}{s} - z \right) + \frac{x \cdot r}{s} \bmod q$$

$$k_0 = \frac{\left(\frac{\text{Hash}(m)}{s} - z \right)}{2^l} + \frac{x \cdot r}{s \cdot 2^l} \bmod q$$

$$x \cdot \frac{-r}{s \cdot 2^l} = \frac{\left(\frac{\text{Hash}(m)}{s} - z \right)}{2^l} - k_0 \bmod q$$

Снова получаем $a \cdot x = b - c$, где $c = k_0$ много меньше чем и $a = \frac{-r}{s \cdot 2^l}$ и $b = \frac{\left(\frac{\text{Hash}(m)}{s} - z \right)}{2^l}$. Поэтому мы можем использовать один и тот же метод.

1.5 Задание

Сервер генерирует два ключа. Он использует первый ключ для подписи 80 сообщений вариантом алгоритма со статистическим отклонением в верхних битах, а второй для подписи 80 сообщений алгоритмом с отклонением в нижних битах. Этого количества подписей должно быть более чем достаточно, чтобы восстановить ключи (степень отклонения можете посмотреть в коде ниже). Ваша цель - восстановить ключи и подписать сообщения 'give' и 'flag' как и в предыдущем задании. Рекомендую сначала написать функцию для решения проблемы для обобщенных $\vec{a}$ и $\vec{b}$, а потом написать преобразование $\text{Hash}(m), r, s$ в a, b . Так Вы сможете переиспользовать код. Также не забывайте, что Вы можете симулировать свой сервер для тестирования (и менять там количество подписей, степень отклонения и другие параметры).

Удачи!

```
[ ]: import random
import time
```

```

from Crypto.Util.number import bytes_to_long
import hashlib
#Это те же классы, которые использует сервер
class DSA:
    def __init__(self,g,p,q):
        self.p=p
        self.g=g
        self.q=q
        self.x=random.randint(1,q-1)
        self.y=pow(g,self.x,self.p)

    def get_k(self):
        raise NotImplementedError()
    @staticmethod
    def message_to_hashnum(m):
        if not isinstance(m,bytes):
            raise TypeError()
        return bytes_to_long(hashlib.sha256(m).digest())
    def sign(self,m):
        r=0
        s=0
        while r==0 or s==0:
            k=0
            while k==0:
                k=self.get_k()
            r=pow(g,k,self.p)%self.q
            s=((DSA.message_to_hashnum(m)+(self.x*r)%self.q)%self.
↪q)*pow(k,q-2,self.q))%self.q
            return (r,s)
    def verify(self,m,r,s,otherY):
        if r==0 or s==0: return False
        w=pow(s,self.q-2,self.q)
        u1=(DSA.message_to_hashnum(m)*w)%self.q
        u2=(r*w)%self.q
        res=(pow(self.g,u1,self.p)*pow(otherY,u2,self.p)%p)%q
        return res==r

class MSBBIasedDSA(DSA):
    def get_k(self):
        return random.randint(0,self.q>>32)

class LSBBIased_DSA(DSA):
    def get_k(self):
        return random.randint(0,self.q-1)|((1<<32)-1)

```

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1351))

    def recv_until(self, symb=b'\n>'):
        """Получаем сообщения от сервера, по умолчанию до следующего,
        ↪приглашения"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self, show=True):
        print ("Пожалуйста, подождите. Создание подписей может занять,
        ↪некоторое время...")
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Ошибка декодирования юникода. Попробуйте,
            ↪переподключиться к серверу')
            return (None, None)
        if show:
            print (data)
        p=int(re.search(r'(?<=p=)\d+', data).group(0))
        q=(p-1)//2
        g=int(re.search(r'(?<=g=)\d+', data).group(0))
        y1=int(re.search(r'(?<=y1=)\d+', data).group(0))
        y2=int(re.search(r'(?<=y2=)\d+', data).group(0))
        all_tuples=re.findall(r'(?<=\\(m,r,s\\)=)\\([a-
        z]+,\\d+,\\d+\\)', data)
        top_biased=[]
        for i in range(0,80):
            spl=all_tuples[i][1:-1].split(',')
            top_biased.append((spl[0].
            ↪encode(),int(spl[1]),int(spl[2])))

        low_biased=[]
        for i in range(80,160):
            spl=all_tuples[i][1:-1].split(',')

```



```

        low_biased.append((spl[0].
↪ encode(),int(spl[1]),int(spl[2])))
        print ('Параметры задания получены')
        return (p,q,g,y1,y2,top_biased,low_biased)

    def checkSolution(self,rg,sg,rf,sf, show=True):
        self.s.sendall((str(rg)+' '+str(sg)+' '+str(rf)+' '
↪ '+str(sf)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Ошибка декодирования юникода. Попробуйте_
↪ переподключиться к серверу')
            return None
        if show:
            print (data)
        if data.find('flag')!=-1:
            return True
        else:
            data=self.recv_until(b'>')
            try:
                data=data.decode()
            except UnicodeDecodeError:
                print ('Ошибка декодирования юникода. Попробуйте_
↪ переподключиться к серверу')
                return None
            if show:
                print (data)
            return False
    def __del__(self):
        self.s.close()

vs=VulnServerClient()
(p,q,g,y1,y2,msb_biased,lsb_biased)=vs.getChallenge(show=False)

```

```

[ ]: bias= # Посмотрите в коде
a_vector=[]
b_vector=[]

# Преобразуйте (H(m),r,s)-> (a,b) для случая MSB (отклонение в_
↪ верхних битах)
# H = DSA.message_to_hashnum

def SolveForAB(a_vector,b_vector,bias,g,y,p,q):
    # Заполните матрицу, используйте LLL, ищите решение в новом базисе
    return x

```

```
x1=SolveForAB(a_vector,b_vector,bias,g,y1,p,q)
```

```
[ ]: a_vector=[]  
b_vector=[]  
  
# Преобразуйте  $(H(m), r, s) \rightarrow (a, b)$  для случая LSB (отклонение в нижних   
↪ битах)  
  
x2=SolveForAB(a_vector,b_vector,bias,g,y2,p,q)
```

```
[ ]: lDSA=LSBBIased_DSA(g,p,q)  
lDSA.x=x1  
lDSA.y=y1  
(r_g,s_g)=lDSA.sign(b'give')  
lDSA.x=x2  
lDSA.y=y2  
(r_f,s_f)=lDSA.sign(b'flag')
```

```
[ ]: vs.checkSolution(r_g,s_g,r_f,s_f)
```

```
[ ]:
```